Filoubatir Fadel CSC4760 (UG)
Problems (1,2,5,6)

- Problems Hand Coded (P1A.cpp, P2A.cpp, P5A.cpp, P6A.cpp)
- Problems Vibe Coded (P1B.cpp, P2B.cpp, P5B.cpp, P6B.cpp)

Time

- P1A.cpp 60 minutes
- P1B.cpp 15 minutes
- P2A.cpp 90 minutes
- P2B.cpp 20 minutes
- P5A.cpp 90 minutes
- P5B.cpp 10 minutes
- P6A.cpp 120 minutes
- P6B.cpp 15 minutes
- Total 420 minutes (7 hours. I didn't have timer in back round so I gave it an estimate this why number look nice I could have spent 13 and I round to 15 or 63 and round to 60)

P1B.cpp
Did the AI use the correct binary tree pattern? Yes it did.
How did it handle the non-divisible N/P case? It handled the non-divisible N/P case correctly.
Were there deadlock risks in the AI's send/receive ordering? It didn't generate any deadlock risks.
Compare your timing tables—any measurable difference in performance? It is hard to tell which one produces a better result. Mine came out faster but not by a major amount. I would say that they are more equal in performance.
Which solution would you trust for a production HPC application, and why? I would trust my code more than chatgpt code because mine does produce runtime errors.

P2B.cpp
Did the AI use the T+1padding correctly? Yes it did.
Did it handle the out-of-bounds guard? Yes it did.
Compare measured execution times: does the tiled version outperform the naive one on your hardware? The vibed code from chatgpt outpeforms my hand written code because it records and stops then using the whole GPU.
Relate the performance difference to the coalescing and bank-conflict concepts from Midterm #2 The kernel used in midterm is slow because threads write different rows causing issues with memory, but the kernel fixes this by staging data through shared memory so both reads and writes to global memory are coalesced.

P5B.cpp

Did the AI place `cudaDeviceSynchronize()` correctly? Yes it did.

Did it handle the scatter/gather buffer sizes correctly? Yes it did.

Any out-of-bounds issues? No issues

Compare with HW#3 Problem #4: how much additional code was needed to “MPI-ify” your original single-process CUDA kernel? I am still doing Hw3. I got an exation on it and haven't made much progress so I can’t really compare.

P6B.cpp

Did the AI correctly compute the scatter distribution indexing? No. The AI acknowledged the scatter distribution formula in a comment but never actually implemented it.

Did it correctly use `Kokkos::fence()` before MPI operations? No, because it is not using scatter distribution it heads to MPI gather but never touches kokkos even though kokkos::fence() is implemented the right way.

Did it use `view.data()` appropriately? No

Explain in your own words why the scatter distribution is more complex to implement than the linear distribution?

Linear distribution you have an array with confcutive chunks all in order 0= 1st chunk 1=2nd chucnk 2=3rd chucnk and so on while scatter distribution uses a more complex pattern 0=1st chunk 2=2nd chucnk c4=3rd chucnk.